Contents

AI Coding Cost Optimisation **1**
Summary . . . 1
Executive summary . . . 1
The problem . . . 2
When not to use AI . . . 2
Token efficiency . . . 2
Open Brain: the memory layer . . . 6
Machine-specific setup . . . 6
The workflow . . . 6
Tools that do the work for you . . . 6
What else to consider . . . 9

AI Coding Cost Optimisation

A practical guide to cutting AI coding costs by 90 per cent without sacrificing quality.

July 2026

Summary

Most developers overspend on AI coding by 5-10x. They use expensive models for trivial tasks, send entire codebases when they need one file, and pay for verbose output that adds no value. Four free, open-source tools — RTK, Caveman, Ponytail, and 9Router — fix this automatically. Installed in five minutes, they compress tool output by 60-90 per cent, agent prose by 65 per cent, and code volume by 54 per cent. Combined with free models from OpenRouter, the monthly cost drops from USD 100-300 to EUR 15-20. The quality does not suffer. The discipline does not change. The tools do the work.

Executive summary

1. **Install RTK.** One command, 60-90 per cent savings on tool output tokens. Biggest single win.
2. **Install Caveman.** One command, 65 per cent savings on agent output tokens.
3. **Install Ponytail.** One command, 54 per cent less code generated.
4. **Use free models for 80 per cent of work.** Reserve paid models for architecture, security, and production.
5. **Enable prompt caching.** Keep static content at the start of every prompt. Do not rearrange it between sessions.
6. **Send only what matters.** Use `@file` references. Be specific about line numbers. Omit unchanged code.
7. **Batch related work.** One request for three bugs, not three requests for one bug each.
8. **Set up Open Brain.** Forty-five minutes of setup saves 15-40 minutes per day of context reloading.
9. **Adopt model cascading.** Free first, paid only on failure.

10. **Use 9Router.** Automatic provider fallback, built-in compression, never stop coding.

Total monthly cost: EUR 15-20. Previous spending: USD 100-300. The tools are free. The discipline is not.

The problem

AI coding assistants burn money fast. At heavy usage rates—roughly 13.5 million tokens per hour—a developer can blow through USD 50 worth of credits in under a day. Most of this spending is wasteful: re-loading context that the system already knows, using expensive models for trivial tasks, and failing to exploit free alternatives that have closed the quality gap to within 5-15 per cent of premium offerings.

The solution is not to spend less time with AI. It is to spend smarter.

When not to use AI

AI is not always the right tool. Using it for every task wastes tokens and time.

Skip AI for: one-line config changes; tasks you already know how to do; debugging when you have not read the error message; exploratory work where understanding the system yourself matters more than delegating it.

Use AI for: multi-file changes where pattern consistency matters; boilerplate generation; code review; tasks outside your primary language or framework.

The cheapest token is the one you never send.

Token efficiency

1. Prompt caching

Most providers cache the beginning of a prompt automatically. If your system instructions and file context appear first, they are served from cache on subsequent requests at 90 per cent lower cost.

How: Put static content at the start of every prompt. Do not rearrange context between sessions. The cache key is positional—moving content breaks it.

2. Context window management

Sending an entire repository when you need one file is like hiring a moving company to carry a book.

Bad: "Here's my codebase: [paste 50 files]" — 10,000 tokens.

Good: "Fix the bug in `@src/auth/login.ts` line 42. The session cookie is not being validated." — 2,000 tokens.

How: Before each request, ask: Am I sending only what the model needs? Use `@file` references. Omit unchanged files. Be specific about line numbers and symptoms.

3. Response length control

Models generate until they run out of tokens or hit a limit. Setting `max_tokens` based on task complexity prevents waste.

Task	max_tokens
Simple fix	500
Code review	1000
Feature plan	2000
Complex refactor	4000

In OpenCode, add to `~/.config/opencode/opencode.json`:

```
{
  "agent": {
    "build": {
      "max_tokens": 2000
    }
  }
}
```

4. Structured output

Request JSON or YAML responses instead of free-form text. Structured output uses 30-50 per cent fewer tokens because the model does not generate filler sentences or explanatory prose. It also eliminates the token cost of parsing free-form responses.

How: Append "Respond as JSON only, no explanation" to your prompt. Use schema validation to catch errors automatically.

5. Model cascading

Try the cheapest model first. Escalate only when quality is insufficient.

When to escalate: - Complex architecture: DeepSeek-V4-Flash (USD 0.09 per MTok) - Security features: Qwen3-Coder-Flash (USD 0.20 per MTok) - Production deployment: MiMo-V2.5-Pro (USD 0.43 per MTok)

6. Failure detection

AI models hallucinate confidently. You need verification systems, not trust.

Hard signals (unambiguous failure): - Build fails. - Tests fail. - Linter catches errors. - Type checker rejects the code.

These are binary. The model failed. Escalate.

Soft signals (suspicious output): - Cross-model review catches issues. - Self-consistency check: ask the same question twice. If the answers diverge, the model is uncertain. - Hallucinated APIs: the model references functions or parameters that do not exist. - Overly confident tone. Models that say "this is definitely correct" are often wrong.

The verification gate:

1. Free model produces output.
2. Automated verification runs: build, lint, test.
3. If all pass, accept. If any fail, escalate.
4. On escalation, a different free model reviews and attempts to fix.
5. If that also fails, escalate to paid model.

A free model that passes all tests is more trustworthy than a paid model that fails them.

7. Choosing the right verification

Match the verification depth to the risk.

Task type	Risk	Verification approach
Typo fix	Trivial	Visual scan
Simple bug fix	Low	Lint + existing tests
New feature (single file)	Medium	Build + lint + targeted tests
New feature (multi-file)	High	Full build + lint + all tests + cross-review
Security change	Critical	Full build + lint + all tests + manual review
Database migration	Critical	Full build + lint + all tests + rollback plan

The rule of thumb: if the change could break production, verify it like it will. If it cannot break anything, skip verification and move on.

8. Worked example

A free model is asked to add a `POST /api/users` endpoint to an Express.js application. The task is high-risk: multi-file change, new database interaction, new API contract.

Step 1: Plan. Free model generates the plan. A different free model reviews it and catches a missing rate-limiting middleware.

Step 2: Implement. Free model writes three files following existing patterns.

Step 3: Verify. `make build` passes. `make lint` passes. Three of 12 tests fail: wrong status codes, missing error handling, and a password hash leak. Hard signal — do not commit.

Step 4: Fix. Model fixes the three failures. Re-run: all 12 tests pass.

Step 5: Review. The `@pr-review` agent finds that the password hash is returned in the error path for duplicate emails — something the tests missed. Fix and re-verify.

Step 6: Commit. All checks pass. Commit.

Stage	What was caught	Caught by
Plan review	Missing rate limiting	Cross-model review
Implementation	Wrong status codes, missing error handling, password leak	Automated tests
Code review	Password hash in error response	Cross-model review (security)

Without verification, all three issues would have reached production. Time cost: roughly three minutes. A paid model would have taken the same time but cost USD 0.50-2.00 instead of nothing.

9. Batching

Three separate bug-fix requests burn three sets of context-loading tokens. One request covering all three bugs saves 40-60 per cent.

Bad: Three requests: "Fix login.ts", "Fix register.ts", "Fix reset-password.ts".

Good: One request: "Fix these three bugs: login.ts line 42 (session cookie), register.ts line 15 (missing validation), reset-password.ts line 28 (token expiry)."

10. Local preprocessing

On a desktop with a 2080 Ti, Ollama runs Qwen 2.5 Coder 7B at 30-50 tokens per second. Use it for code formatting, simple refactors, and test generation. Do not use it for complex architecture, multi-file changes, or security decisions.

11. Agent specialisation

Different models have different strengths. Using the same model for everything wastes its strengths and exposes its weaknesses.

Task	Model	Why
Planning	DeepSeek-V4-Flash Free	Fast, good at structure
Coding	DeepSeek-V4-Flash Free	Reliable, free
Reviews	Nemotron 3 Ultra Free	Different architecture, catches different bugs
Security	Qwen3-Coder-Flash (paid)	Stronger reasoning

Cross-model review is not optional. It catches semantic errors that same-model review misses.

12. Streaming for faster feedback

Enable streaming responses. You see output as it generates, not after a 30-second wait. This cuts perceived latency by 80 per cent and lets you spot bad output early — before the model finishes generating 4,000 tokens of wrong code.

How: Most tools enable streaming by default. If yours does not, check the settings.

13. Local linting before AI

Run your linter before sending code to AI. A model that receives clean code produces cleaner output. A model that receives code with lint errors often reproduces or compounds them.

How: `make lint` before every AI request. Ten seconds of local linting saves minutes of AI-generated fixes.

Open Brain: the memory layer

Context reloading is the single largest source of waste. Every new session re-explains project structure, conventions, and recent decisions. Open Brain — an open-source system that gives every AI tool the same persistent memory via vector search and MCP — solves this. You store decisions, patterns, and context once; every future session starts with the AI already knowing your project.

Setup time: 15-45 minutes depending on cloud or offline deployment. **Detailed guide:** see `open-brain-guide.md`.

Machine-specific setup

Chromebook

No GPU, no local models. Use OpenRouter exclusively. Set free models as default in the global config. Set up Open Brain for persistent memory.

Budget: USD 0-5 per day.

Desktop (i9-9900K / 2080 Ti)

Full GPU available. Use local Ollama for most work, OpenRouter as fallback. Set up Open Brain for persistent memory.

Budget: EUR 13 per month electricity plus USD 0-10 API.

Dual machine

Use the Chromebook for mobile work with free API models. Use the desktop for offline and privacy-sensitive tasks with local models. Open Brain syncs knowledge between both.

Combined cost: EUR 15-20 per month.

The workflow

1. **Start of session:** AI searches Open Brain for relevant context. No re-explaining.
2. **Planning:** Free model writes the plan. Different free model reviews it.
3. **Implementation:** Free model writes code. Send only relevant files via `@file` references.
4. **Verification:** Free model runs build/lint/tests. Escalate to paid model only on failure.
5. **Capture:** Store decisions, patterns, and debugging notes in Open Brain for next time.

Per-feature cost drops from USD 5-20 to effectively zero.

Tools that do the work for you

The techniques above are habits. These tools automate them. They are ordered by impact: the first saves the most tokens for the least effort.

Priority 1: RTK — compress tool output (68,000 GitHub stars)

Every `git diff`, `grep`, `ls`, and test runner dumps raw output into the LLM context. RTK intercepts these commands and compresses the output before it reaches the model. In a typical 30-minute session, it cuts input tokens by 80 per cent.

It is a single Rust binary with zero dependencies. Install it, run `rtk init -g`, and restart your AI tool. Every shell command is then silently rewritten to its compressed equivalent. `git push` returns "ok main" instead of 15 lines of progress output. `cargo test` shows only failures instead of 200 lines of passing tests.

Savings: 60-90 per cent on input tokens from tool output. **Cost:** Zero. **Effort:** One command to install.

```
brew install rtk && rtk init -g
```

Priority 2: Caveman — compress agent output (84,000 GitHub stars)

RTK compresses what goes *into* the model. Caveman compresses what comes *out*. It injects a prompt that makes the agent reply in terse fragments instead of verbose prose. The same technical answer, 65 per cent fewer output tokens.

Install once, and every session starts in caveman mode. Turn it off with "normal mode" if you need full prose for a complex explanation.

Savings: 65 per cent on output tokens. **Cost:** Zero. **Effort:** One command to install.

```
curl -fsSL https://raw.githubusercontent.com/JuliusBrussee/caveman/main/install.sh | bash
```

Priority 3: Ponytail — write less code (74,000 GitHub stars)

Caveman makes the agent *talk* less. Ponytail makes it *write* less code. It injects a "lazy senior dev" ruleset: before writing code, the agent checks whether a stdlib function, a native HTML element, or an existing dependency already does the job. The result is 54 per cent fewer lines of code on average, with no loss of safety.

Where Caveman is about output tokens, Ponytail is about code volume. The two stack: Caveman compresses the prose around the code, Ponytail compresses the code itself.

Savings: 54 per cent fewer lines of code, 20 per cent cheaper. **Cost:** Zero. **Effort:** One command to install.

```
curl -fsSL https://raw.githubusercontent.com/DietrichGebert/ponytail/main/install.sh | bash
```

Priority 4: 9Router — smart provider routing (19,000 GitHub stars)

9Router is a local proxy that sits between your AI tool and your providers. It routes requests through a 3-tier fallback: subscription models first, then cheap paid models, then free models. If your Claude Code quota runs out mid-session, it silently switches to a free provider without interrupting your work.

It also includes RTK compression built in, plus optional "Caveman mode" and "Ponytail mode" — so it can run all three tools through a single proxy.

Savings: Automatic fallback prevents downtime; built-in compression saves 20-40 per cent. **Cost:** Zero. **Effort:** `npm install -g 9router && 9router`.

How they stack

Tool	What it compresses	Savings	Effort
RTK	Tool output (git, grep, tests)	60-90 per cent input	One command
Caveman	Agent prose	65 per cent output	One command
Ponytail	Code volume	54 per cent fewer lines	One command
9Router	Provider routing + all above	20-40 per cent + fallback	One command

All four are free, open-source, and work with Claude Code, Codex, Cursor, Gemini CLI, OpenCode, and 30 other agents. Install all four for maximum effect.

Choosing your AI tool and provider

The tools above save tokens. Your choice of AI *tool* and *provider* determines how much those tokens cost.

AI coding tools — scored comparison (1-5, higher is better):

Tool	Cost control	Model flexibility	Agent quality	Free tier	Ease of use	Effectiveness
OpenCode	5	5	4	5	3	**4.4**
Cline	5	5	3	5	3	**4.2**
Gemini CLI	4	1	3	5	5	**3.6**
Cursor	2	4	4	3	5	**3.6**
Codex	3	1	4	2	5	**3.0**
Claude Code	1	1	5	1	5	**2.6**

How to read this table. Cost control: can you pick cheap models? Model flexibility: how many providers can you use? Agent quality: how well does it code? Free tier: can you use it for nothing? Ease of use: how much setup does it need?

OpenCode and Cline score highest because they let you use any model from any provider, including free ones. Claude Code scores lowest on cost despite having the best agent quality — you are locked into Anthropic's pricing.

Provider aggregators — scored comparison (1-5, higher is better):

Provider	Price	Free tier	Model selection	Reliability	Ease of use	Effective-ness
OpenRouter	5	4	5	5	5	**4.8**
9Router	5	5	4	4	4	**4.4**
Google Vertex	4	5	3	5	3	**4.0**
Kiro AI	5	5	3	2	4	**3.8**

How to read this table. Price: cost per token. Free tier: how much can you use for nothing? Model selection: how many models are available? Reliability: uptime and consistency. Ease of use: setup complexity.

OpenRouter scores highest because it has the widest model selection, transparent pricing, and the most reliable infrastructure. 9Router scores well on price and free tier but adds a proxy layer that can break. Kiro AI offers free Claude but may not last — free tiers from startups are inherently unreliable.

The cost trap: Cursor's tab-completion uses small, cheap models and costs little. But its agent mode (Ctrl+K, chat) uses the same expensive models as Claude Code. Most developers burn money on the agent, not the autocomplete.

The provider trap: Subscriptions (Claude Pro, Cursor Pro) give you a fixed quota that expires. Pay-as-you-go (OpenRouter, 9Router) gives you tokens that roll over and cost less per unit. For heavy users, pay-as-you-go is almost always cheaper.

Recommendation: Use OpenCode or Cline as your tool (free, any model), OpenRouter as your provider (cheapest paid, best free selection), and 9Router as your router (auto-fallback). This combination gives you maximum control over cost with zero subscription lock-in.

What else to consider

Pre-commit hooks. Run lint and type-check automatically before every commit. Catches issues before they reach the AI, saving tokens on both sides.

Git stash for experimentation. When trying a risky approach, `git stash` your work first. If the AI produces garbage, restore in seconds. This encourages experimentation without fear of losing work.

Diff-based edits. Prefer tools that send only the changed lines, not entire files. OpenCode's edit tool sends the diff, not the file. This cuts context tokens by 60-80 per cent for large files.

Conversation management. Start a new conversation for each distinct task. Long conversations degrade model performance and waste tokens re-loading old context. If you have been chatting for more than 20 minutes, start fresh.

Model rotation. Rotate between free models every few requests. This avoids rate limits and exposes you to different model strengths. DeepSeek-V4-Flash for coding, Nemotron for reviews, Gemma for variety.

Prompt templates. Write reusable prompts for common tasks: code review, bug fix, test generation, plan writing. Store them in your project. A good template saves 20-30 tokens per

request and produces more consistent output.

Monitor your spending. Check your OpenRouter dashboard weekly. If you are spending more than USD 20 per month on paid models, you are escalating too often. Free models should handle 80 per cent of work.

Sources: OpenRouter pricing (June 2026), MiMo Code credit documentation, independent model benchmarks (LiveCodeBench, Coding Index, Agentic Index). Benchmark figures are approximate and sourced from public leaderboards at time of writing. Costs are in the currency quoted by each provider; EUR 1 is approximately USD 1.10 at time of writing.